

从 Spec-Driven Development 到 Attractor-Guided Engineering

原创 可逆计算 可逆计算 2026年5月20日 22:05 北京

[nop-chaos-flux](#) 是一个基于 React 19、Zustand 5 和 Vite 8 构建的低代码运行时框架。它受百度 AMIS 启发，但基于创新的设计原理从零开始独立实现，包含 JSON Schema 编译器和 runtime，也有报表设计器和 Flow Designer 等设计工具。

这个项目由单个程序员在2个月的时间内完成初步版本，完全使用 AI 自动化开发。与一般AI辅助开发不同的是，在不断增加功能实现的过程中，项目并没有出现质量劣化，反而模块边界越来越清楚，文档分类越来越稳定，测试和审计越来越能抓住真实问题，代码质量在不断改进。

我最近在公司内部做了一些内部分享，把这套实践命名为 Attractor-Guided Engineering（吸引子引导工程，简称 AGE）。后来在代码评审中发现很多人并没真正理解。一些人的操作方式接近随意的 vibe coding，另一些人则采用类似 OpenSpec 的 spec driven 开发方式，整体偏向任务驱动、逐特性开发。随着工作量累积，代码中的不一致性在稳步累积，偏离架构设计的情况成为常态。

本文剖析 AGE 与 spec driven 开发的区别，以及普通业务项目能从 [nop-chaos-flux](#) 的大规模 AI 工程实践中学到什么。

核心层级

AGE 的核心是如下概念层级：

状态空间 → 吸引子 → 轨迹 → 控制

- **状态空间**：系统在当前约束下所有可能演化到的实现状态
- **吸引子**：系统在长期演化中反复被拉回的稳定结构
- **轨迹**：每一轮生成、验证、纠偏之后真实留下来的演化路径
- **控制**：通过局部信号持续影响轨迹的各种机制

仓库在现有约束下可能演变出的所有代码、文档和测试组合，构成了状态空间；人、AI、review、CI 以及文档更新等持续作用，构成了演化规则；二者叠加形成的 live repo 历史，就是轨迹；而 attractor，则是系统在长期迭代中反复被拉回的那个稳定结构。

AI 大规模开发本质是一个动力系统的受控收敛问题。AI 展开状态空间的速度极快，关键不是到处增加 guardrail，而是要先想清楚：系统长期应该被拉回到什么结构。

吸引子不是被完整写出的终点，不是路线图，也不是"允许活动的范围"。它是少量高阶约束隐式定义出的结构：局部实现可以变，整体会被这些关系拉回同一类形态。

在 `nop-chaos-flux` 中，最先定义 attractor 的不是 plan、lint、test、audit，也不是某个 spec 文件，而是 `docs/architecture/` 下带 precedence 的 architecture docs。下文中，这种带 owner 和 precedence 的 architecture docs 就叫 owner docs。

`docs/architecture/README.md` 中明确写明：`docs/architecture/` 是当前 final-state architecture baseline，`flux-design-principles.md` 是 governing-principles anchor，`frontend-programming-model.md` 拥有顶层 primitive 和 core-boundary precedence，其他 normative docs 在各自主题内拥有 local precedence。

这些文档不枚举所有正确实现。它们像方程一样定义一个稳定结构：有哪些原语，哪些依赖方向合法，哪些 owner 边界不能破，哪些旧模式已经不属于正确状态空间。它们先定义了可持续存在的状态簇，后面的实现、测试、计划和审计才有参照。

Plan、verification、audit、logs、bugs、testing 都在后面。它们不是 attractor 本身，而是让系统轨迹持续贴近 attractor 的 harness。

Flux/AGE 不是以 spec 演化为中心，而是以 attractor 定义和轨迹收敛为中心。

真实研发历史

"架构文档定义吸引子"乍听起来还是像一般的文档治理，但是如果仔细查看 [docs/logs/](#) 里的实际研发记录就能看出区别。

比如说，[docs/plans/371-deep-audit-2026-05-19-owner-routing-plan.md](#) 是个典型入口。它不直接改代码，而是把 2026-05-19 deep audit 的 64 个 retained findings 收敛成一份 owner-routing baseline。每条 finding 都有唯一 owner bucket、优先级、successor plan 和 owner-doc obligation。没有 ownerless、没有 multiply-owned、没有静默降级成 vague follow-up。

它的作用是先冻结一轮偏离该怎样收口，后续的执行计划再把局部轨迹闭合到具体结果面。[docs/plans/382-deep-audit-2026-05-19-table-and-crud-owner-state-and-event-contract-plan.md](#) 收口 table/CRUD owner state 和 event payload: [Current Baseline](#) 声明显式空数组 owner state 与 event payload 的 live 状态，执行项修复 empty-array fallback, focused tests 证明结果，[docs/components/table/design.md](#) 和 [docs/components/crud/design.md](#) 同步 owner contract, 最后 repo-wide 的 `pnpm typecheck`、`pnpm build`、`pnpm lint`、`pnpm test` 全过，独立 closure audit [ses_1bd9ed593ffeVpkho41b4wPR6p](#) 记录 `Verdict: acceptable`。

[docs/plans/388-deep-audit-2026-05-19-form-tree-widget-accessibility-plan.md](#) 展示了另一种 closure: input-tree 和 tree-select 的可见节点 roving focus、[ArrowUp / ArrowDown / Home / End](#) 导航、loading [aria-busy / aria-describedby](#) 都有 focused proof，但计划明确裁定 `No owner-doc update required`。说明 plan 不只是做事清单，也负责判断本次变化是否改动了 owner-doc baseline。

[docs/plans/400-deep-audit-2026-05-19-test-harness-reliability-plan.md](#) 和 [docs/bugs/62-e2e-shared-websocket-error-suppression-fix.md](#) 展示了 memory harness 的作用。问题不是某个业务功能失败，而是 E2E shared fixture 过滤范围太宽，把真实的 transport/runtime failures 也过滤掉了。修复不单改代码，还新增 [tests/e2e/fixtures-hard-gate.spec.ts](#) 证明现在会失败，并用 bug note 记录规则：如果未来测试确实预期 WebSocket failure，必须用 per-test allowance，不能恢复 fixture-wide suppression。

[docs/logs/2026/05-19.md](#) 和 [docs/logs/2026/05-20.md](#) 不是流水账。它们记录每个 closure slice 的 focused proof、owner-doc 裁定、repo-wide gates、independent audit 和 full-green baseline。

这些材料形成一条真实链路：owner docs 定义长期结构，audit 发现偏离，plan 路由并冻结 owner，focused proof 证明局部结果，owner-doc 裁定是否更新，logs/bugs/testing 保存跨 session 记忆，closure audit 从 live repo 重新判定完成状态。

这就是 AGE 里的 harness。目标不是让每次任务显得更完整，而是让系统轨迹持续回到 attractor 附近。

OpenSpec 的结构强在哪里

OpenSpec 是一个有代表性的 spec driven 开发框架，它把一类工作结构化了：

- `openspec/specs/` 保存当前能力的行为规格。
- `openspec/changes/` 保存拟议变更，包含 `proposal.md`（为什么改、改什么、影响什么）、`design.md`（必要时记录技术设计）、`tasks.md`（实施 checklist）。
- `changes/<name>/specs/` 用 `ADDED/MODIFIED/REMOVED/RENAMED` 记录规格 delta。
- `archive` 通过固定规则把 delta 应用回 main specs。

这套机制降低了规格更新成本。OpenSpec 不需要每次让 AI 自由理解整份需求再随意改文档，而是用固定 section、requirement header matching、delta application，把结构化变更合并回主规格。这种做法很适合行为规格明确、需要可解析、可归档、可回写的场景。

OpenSpec 的限制在哪里

OpenSpec 的 spec 格式（Requirement、Scenario、SHALL/MUST、delta sections）对工具解析友好，但作为通用项目知识组织方式不够灵活。

很多仓库知识不天然长成 requirement/scenario：

- 架构层级和 precedence。
- 术语误读历史。
- 外部研究材料（PPT、讨论记录）。
- 某次源码审计的分析结论。
- 人工测试发现的现象。
- 复杂 bug 的排除路径。
- 一次 plan 为什么不能关闭。

如果都要先翻译成 spec/change 才能获得正式位置，转换成本很大，也会损失原始语境。外部文档、讨论记录、分析报告、bug 复盘，本来可以直接成为仓库记忆。强行转成 requirement/scenario，反而容易把信息压扁。

执行验证方面，两者反映了不同的工具哲学。OpenSpec 的 `tasks.md` 是 checklist，`/ops x:verify` 是可选的 agent 验证技能（需手动启用 expanded workflows）。`archive` 会检查 task 状态并提示风险，但用户确认后仍可继续——严格 closure 留给团队自行约定。

Flux plan 则把独立 closure audit 内置为必经环节：必须从当前基线开始，写清 `Goals`、`Non-Goals`、执行项（每项标注 `Fix / Decision / Proof`）、`Closure Gates`；标记 `complete`

d 前，必须有独立子 agent 或独立 reviewer 回到 live repo 做 closure audit。 [docs/plans/361-slot-contract-host-manifest-and-owner-doc-closure-plan.md](#) 把当前基线、finding 所有权矩阵、执行阶段、closure gates、被裁定的 deferred 项、独立审查证据放在一起。这不是 checklist，而是严格定义了关闭条件并要求独立审计的执行合同。它强调的不是做了什么，而是做到什么程度才算真的落地。

为什么 spec-driven 容易变成任务导向

Spec-driven 的默认结构很容易把人和 AI 的注意力拉回“这次 change 怎么完成”。

一次 change 通常有 proposal、design、tasks、delta specs。这比口头需求好得多，但天然围绕一次变更组织：为什么改、改什么、怎么做、做完哪些 checklist。时间一长，spec-driven 容易变成一种更正式的任务派发系统。

这种任务导向有一个隐蔽风险：spec 更新了，tasks 勾了，archive 做了，AI 和人都会获得强烈的完成感。但系统是否真的更接近长期结构，不是由这次 change 自己证明的。它要回到架构文档、live repo、测试、日志和独立审计里看。

如果试图把 spec 长期维护到 Flux 这种精度，成本通常更高。Flux 维护的不是单一规格树，而是一组职责不同的仓库记忆：架构文档定义 attractor，plans 定义局部轨迹收口，logs 记录演化，bugs 保存复杂缺陷诊断，testing 保存人工发现，analysis 保存研究判断，audit 负责回看 live repo。全部压进 spec/change 结构，会产生持续转换成本。

Spec-driven 可以作为 AGE 的一个局部 harness，用来管理行为规格演化。但它不应该替代架构文档，也不应该替代 plan closure、logs、bugs、testing 和 audit 这些各自独立的仓库记忆。

单独要看 owner doc 怎么办

这能看出两种方法的根本差别。

如果在 Flux 里只是要查 renderer runtime 的当前 owner contract，可以直接从 [docs/index.md](#) 路由到 [docs/architecture/renderer-runtime.md](#) 和相关 reference。你不需要创建一个 change，不需要 proposal，不需要 tasks，也不需要把问题改写成 spec delta。

如果只是一次研究，可能写入 [docs/analysis/](#)。如果只是人工测试发现，先进入 [docs/testing/](#)。如果是复杂 bug，进入 [docs/bugs/](#)。如果是一次执行收口，才写 plan。稳定基线变化，才回到 owner doc。

Flux 的文档组织是自由的，但不是随意的。自由指的是：它不强迫所有知识先进入同一套 artifact workflow。约束指的是：每类材料都有明确职责，不能乱抢事实地位。

OpenSpec 的组织更结构化。它适合把拟议行为变更放进 change package，再把规格 delta 回写到 main specs。但通用仓库知识路由不是它默认主线要解决的问题。

Tasks 不是 Plan

OpenSpec 的 `tasks.md` 不对应 Flux 的 `plans`。

`tasks.md` 最多对应 Flux plan 里的执行清单片段，或者当前会话的 todo。它能帮助 AI 不漏步骤，但不回答这些问题：

- 当前 live repo 基线是什么？
- 本次明确不做什么？
- 哪些证据能证明真的完成？
- 哪些缺陷不能降级成 follow-up？
- 谁独立复核完成状态？

Flux plan 的价值不在于“任务更多”，而在于它是一份局部轨迹的关闭合同。

所以 tasks 对 AI 有必要，但 tasks 不是事实源。 - [x] 只说明执行者声称某项完成了。真正的证明要回到当前代码、tests、owner docs 和独立审查。

变化来源不止 spec change

OpenSpec 的路径很清晰：围绕 specs 和 changes 组织行为规格演化。

Flux 的变化来源更广：

- 用户提出的新需求。
- 阅读源码发现的架构漂移。
- audit 发现的 owner boundary 问题。
- testing issue 暴露的交互问题。
- bug note 记录的历史回归风险。
- 外部 research 或 analysis 得出的设计判断。
- plan closure audit 推翻了此前“已完成”的判断。

这些来源不一定都应该先变成 spec delta。可以先进入 analysis、testing、bugs、logs、plans，或者直接更新架构文档。是否需要最终变成行为 contract，取决于它是不是长期基线的一部分。

这就是 Flux 文档组织自由度的意义。不同来源的信息先停在不同位置，而不是全部塞进一个规格演化通道。

文件进文件出为什么重要

我在分享中多次强调过，AI 驱动开发的一个最佳实践是文件进、文件出。这不只是文档洁癖——attractor 和 harness 需要承载物。

文件进：输入不要只留在聊天窗口里。哪怕内容散乱，也先写到需求文件、分析文件或计划文件里，再在 OpenCode 中 @ 这个文件。

文件出：输出不要只打印在窗口里就结束。重要结论、分析、计划、测试记录、bug 诊断、架构约束，都写入 docs/，按职责分类保存：

- 架构规则 → docs/architecture/
- 变更收口 → docs/plans/
- 执行轨迹 → docs/logs/
- 人工测试发现 → docs/testing/
- 复杂缺陷诊断 → docs/bugs/
- 研究判断 → docs/analysis/

重点不是“多写文档”。输出一旦分类落盘，就进入了仓库记忆。聊天窗口是临时上下文，文件才是仓库记忆。

代码是当前实现事实，不是唯一事实

代码是当前实现的事实源。类型和测试保护当前行为。带 precedence 的 architecture owner docs 定义 attractor——系统长期应该收敛到的结构。Logs、bugs、analysis、testing 记录轨迹和外部化记忆。

这些并不冲突。冲突发生在两种情况下：

- 文档开始复述易腐烂的实现细节。
- 文档的完成状态被当成代码已经正确的证明。

OpenSpec 的 behavior-first boundary 其实也在避免第一个问题：behavior spec 应该写可验证行为，不写内部实现细节。Flux 进一步强调第二个问题：即使文档写的是正确行为，也不能替代当前仓库验证。真正关闭时必须回到代码、测试、owner docs、logs 和审计证据。

真正的差别

OpenSpec 解决的是：如何用结构化 specs/changes/delta/archive 组织行为规格的更新，关注的是规格的演化流

AGE 解决的是：AI 高速展开状态空间时，系统如何先定义 attractor，再通过 plan、verification、audit、logs、bugs、analysis、testing 等 harness，让轨迹持续收敛。

两个工具解决的问题层级不同。OpenSpec 的强项是结构化规格 workflow。Flux/AGE 的强项是以带 precedence 的 architecture owner docs 定义吸引子，再用自由但有职责边界的文档路由和严格 closure harness 让仓库长期收敛。

从 spec-driven development 到 Attractor-Guided Engineering，不是从"写 spec"走向"不写 spec"，而是从"把变更组织成规格更新"，走向"定义系统应向哪里长期收敛，并让每一轮 AI 生成都被这个结构持续拉回"。

普通项目能直接用哪几条

不需要照搬 Flux 的完整治理。

第一条，文件进文件出。 输入先写到文件再 @ 给 AI；输出写入 docs/，不要只留在聊天窗口。这一步花不了几分钟，但它让 AI 不再依赖上下文记忆，而是有了可回溯的仓库记忆。

第二条，给核心业务写一份架构文档，定义业务吸引子。 不写代码导游，只写事实来源、状态转换、所有权边界和常见误解。

第三条，复杂变更写轻量 plan。 包含当前基线、目标、不做什么、执行清单、证明项和关闭条件。完成前由独立会话或独立 reviewer 复核。

第四条，把反复出现的问题变成仓库记忆。 人工问题进 testing，复杂 bug 进 bugs，研究结论进 analysis，高频错误进审计脚本。

规格更新是必要的一层。系统收敛还需要 attractor。

nop-chaos-flux 已开源：

- GitHub: <https://github.com/entropy-cloud/nop-chaos-flux>
- Gitee: <https://gitee.com/canonical-entropy/nop-chaos-flux>

喜欢作者

Nop平台 · 目录

上一篇

NOP Chaos Flux 架构演变史：从 AMIS 重写到现代低代码运行时

下一篇

超越Harness Engineering: AGE 应用开发模板介绍

